

Online Retail II — Müşteri Yaşam Boyu Değeri (CLV) Tahmin Sistemi

Bilgisayar Mühendisliği — Büyük Veri Analizine Giriş

Dönem: 2025-2026 Bahar

Öğrenciler: Tuna Kömür, Kader Kırççek, Osman Aldemir, Yaren Güner

Repo: <https://github.com/TunaKomur/big-data-online-retail-pipeline>

Tarih: Mayıs 2026

1. Özet

Bu projede, **Online Retail II** veri seti üzerinde uçtan uca bir büyük veri pipeline'ı kurulmuştur. Apache Kafka ile streaming veri üretimi, Apache Spark ile veri işleme, Delta Lake ile çok katmanlı depolama (Medallion mimarisi), Spark MLlib ile 5 farklı regresyon modeli, MLflow ile deney takibi ve Streamlit ile interaktif dashboard geliştirilmiştir.

Problem: Müşterilerin geçmiş davranışlarına dayanarak gelecekteki harcamalarını (Customer Lifetime Value) tahmin etmek — regresyon problemi.

En İyi Model: Linear Regression, $R^2 = 0.66$, RMSE = £1596

2. Problem Tanımı

2.1 İş Problemi

E-ticaret işletmeleri için müşteri yaşam boyu değeri tahmini, **pazarlama bütçesinin optimizasyonu**, **müşteri segmentasyonu** ve **kişiselleştirilmiş kampanyalar** için kritiktir. Geçmişte yüksek harcama yapan veya yapacak müşteriler belirlenip onlara yönelik özel stratejiler geliştirilebilir.

2.2 Teknik Problem

- Tip:** Denetimli öğrenme — Regresyon
- Hedef Değişken (Y):** Müşterinin sonraki 6 ayda yapacağı toplam harcama (£)
- Bağımsız Değişkenler (X):** 10 feature (RFM + davranışsal sinyaller)

2.3 Zamansal Bölümleme

Modelin "geçmişten geleceği tahmin etme" prensibine sadık kalması için veri zamansal olarak bölündü:

- Feature Dönemi (geçmiş %75):** ~18 ay → Feature'lar türetildi
- Target Dönemi (gelecek %25):** ~6 ay → Müşterinin harcaması (Y)

Bu yaklaşım, ML literatüründe **data leakage'i önleyen** doğru metodolojidir.

3. Veri Seti

Özellik	Değer
Kaynak	UCI Machine Learning Repository
İşlem Sayısı	~1.067.371
Tarih Aralığı	01/12/2009 - 09/12/2011
Sütunlar	InvoiceNo, StockCode, Description, Quantity, InvoiceDate, UnitPrice, CustomerID, Country
Boyut	43.5 MB
Format	Excel (.xlsx)

Streaming Simülasyonu: Production senaryosunu yansıtmak için 100.000 işlem Kafka'ya aktıldı. Tam veri seti ile pipeline aynen çalışabilir.

4. Sistem Mimarisi

4.1 Genel Akış

[Excel Veri Seti] ↓ [Python Producer] (Docker) ↓ [Kafka Topic] (Docker — 3 partition) ↓ [Spark Structured Streaming] (Host — Apple Silicon native) ↓ [Delta Lake: Bronze → Silver → Gold] (Host) ↓ [Spark MLlib + MLflow] (Host + Docker) ↓ [Streamlit Dashboard] (Host)

4.2 Servis Mimarisi

Servis	Container/Host	Port	Görev
Zookeeper	Docker	2181	Kafka koordinasyonu
Kafka	Docker	9092, 9094	Mesaj kuyruğu
Kafka UI	Docker	8080	Yönetim arayüzü
Producer	Docker	-	JSON mesaj üretici
MLflow	Docker	5001	Deney takibi
Spark	Host	4040	Veri işleme
Jupyter	Host	8888	Geliştirme
Streamlit	Host	8501	Dashboard

4.3 Apple Silicon Mimari Kararı

İlk yaklaşımda Spark da Docker'da çalıştırılmaya çalışıldı; ancak **Apple Silicon (M2) üzerinde amd64 emülasyonu nedeniyle** Spark Session başlatma sırasında kararsız çalıştı (donmalar, geç başlama). Spark host'ta (ARM64 native) çalıştırıldı. Sonuç: **15x daha hızlı session başlatma** (2.1 saniye vs 30+ saniye).

4.4 Medallion Mimarisi (Delta Lake)

Katman	Görev	Şema
Bronze	Raw streaming — Kafka mesajları ham haliyle	timestamp, kullanıcı_ID, olay_tipi, ilgili_ID, kafka metadata
Silver	Temizlenmiş, parse edilmiş — purchase/cancellation ayrımı	+ invoice_date, quantity, unit_price, total_price, country, description
Gold	Müşteri seviyesi feature tablosu — ML için	10 feature + future_spending (target)

5. Veri İşleme Pipeline

5.1 Faz 1-3: Veri Keşfi ve Streaming

- Pandas ile ilk keşif (1M+ satır)
- Docker Compose ile servis orkestrasyon
- Kafka Producer: her satır → JSON mesaj → topic

5.2 Faz 4: Bronze ve Silver

Bronze (streaming): Kafka'dan structured streaming ile okuma, Delta Lake'e yazma (checkpoint'li, exactly-once semantics)

Silver (batch): Bronze'dan okuma → temizlik:

- Null kontrolü (CustomerID, Description)
- Tip dönüşümleri (timestamp, decimal)
- İptal/alışveriş ayrımı (InvoiceNo'da "C" prefix'i)
- Cancellation tablosu ayrı yazıldı
- Final: %100 dolu (null yok)

5.3 Faz 5: EDA

Silver üzerinde 8 görsel analiz:

- Aylık satış trendi (Kasım-Aralık tepe)
- Saatlik yoğunluk (10:00-15:00 peak)

- Coğrafi dağılım (UK %90+)
- Müşteri harcama dağılımı (sağa çarpık)
- Top 20 ürün, top 10 müşteri
- Sütun bazlı doluluk analizi (%100 dolu)
- İptal analizi (~%2 iptal oranı)

5.4 Faz 6: Feature Engineering

10 müşteri seviyesi feature üretildi:

#	Feature	Açıklama	İş Mantığı
1	recency_days	Son alışverişten gözlem tarihine gün	Yakın alışveriş = sadık
2	frequency	Toplam fatura sayısı	Sık alışveriş = aktif
3	monetary	Toplam harcama	Geçmiş = geleceğin sinyali
4	avg_basket_value	Sepet ortalaması	Pahalı sepet alışkanlığı
5	avg_days_between_purchases	Sıklık göstergesi	Düzenlilik
6	unique_products	Ürün çeşitliliği	Engagement
7	cancellation_count	Geçmiş iptaller	Memnuniyetsizlik
8	active_months	Aktif ay sayısı	Uzun ömür
9	most_active_hour	Yoğun alışveriş saati	Davranış paterni
10	country	Müşteri ülkesi	Coğrafi sinyal

Target: future_spending – Tahmin döneminde toplam harcama

6. Makine Öğrenmesi

6.1 Pipeline

- Outlier temizliği: üst %1 (target ve monetary)
- Train/Test split: 80/20
- Preprocessing: StringIndexer + OneHotEncoder + VectorAssembler + StandardScaler
- 5 model paralel eğitim
- MLflow log: parametreler, metrikler, model artifact'i

6.2 Sonuçlar

Model	RMSE	MAE	R²	Eğitim Süresi
LinearRegression	1596.17	676.89	0.6584	0.24 s
GeneralizedLinearRegression	1596.17	676.89	0.6584	0.08 s
DecisionTree	1855.17	743.38	0.5385	0.54 s
RandomForest	2267.06	854.71	0.3109	0.68 s
GradientBoostedTrees	3250.59	1183.74	-0.4168	6.22 s

6.3 Yorum

Sürpriz Bulgu: Linear Regression ensemble modellerinden (RF, GBT) daha iyi performans gösterdi. Sebep:

- Veri seti büyüklüğü: 317 müşteri eğitim için sınırlı. Ensemble modeller daha çok veriye ihtiyaç duyar.
- Overfitting: RF ve GBT küçük veride aşırı karmaşık kalıyor.
- Basitliğin gücü: Az veride doğrusal modeller daha iyi genelleme yapar.

Tam veri seti (1M+) ile GBT'nin Linear'i geçmesi beklenir. Bu projede pipeline'ın doğru kurulduğunu göstermek için 100K mesaj örnekleme kullanıldı.

6.4 Feature Importance

Random Forest ve GBT modellerinden çıkarılan top 3 feature:

1. **monetary** — En güçlü sinyal (geçmiş = gelecek)
2. **frequency** — Aktivlik göstergesi
3. **recency_days** — Sadakat sinyali

Bu RFM çekirdeği literatürle uyumlu (40+ yıllık e-ticaret modellemesi).

7. MLflow ile Deney Takibi

Tüm modeller için kaydedilen bilgiler:

- **Parametreler:** maxIter, regParam, maxDepth, numTrees, vb.
- **Metrikler:** RMSE, MAE, R², training_time_seconds
- **Artifact'ler:** Eğitilmiş model (yeniden yüklenebilir)
- **Tag'ler:** model_type

Experiment: customer_lifetime_value_prediction

URL: <http://localhost:5001>

Bu sayede deneyler arası karşılaştırma, model versiyonlama ve reproduksiyon mümkün.

8. Dashboard

Streamlit + Plotly ile interaktif arayüz. 4 sayfa:

1. **Ana Sayfa** — Proje özeti, veri istatistikleri
2. **EDA** — Zaman serisi, dağılımlar, kategorik analiz
3. **Customer Analysis** — RFM görselleri, top müşteriler, segmentasyon
4. **Model Comparison** — 5 model performans grafiği (MLflow'dan)
5. **Predictions** — Feature importance, gerçek vs tahmin scatter, residual analizi

Erişim: streamlit run dashboard/app.py → <http://localhost:8501>

9. Karşılaşılan Zorluklar ve Çözümler

9.1 Spark Apple Silicon Uyumsuzluğu

Sorun: Docker'da Spark amd64 emülasyonu donmalara sebep oluyordu.

Çözüm: Spark host'ta native ARM64 olarak çalıştırıldı, 15x performans iyileşmesi.

9.2 MLflow Read-Only Filesystem

Sorun: Model artifact'leri kaydedilemiyor ([Errno 30] Read-only file system: '/mlflow-artifacts').

Çözüm: Artifact path'i /mlruns/artifacts olarak değiştirildi, volume mount düzeltildi.

9.3 setuptools 81 Uyumsuzluğu

Sorun: setuptools 81+ pkg_resources modülünü kaldırdı, MLflow hâlâ buna bağımlı.

Çözüm: setuptools<81 sürümüne pin'lendi.

9.4 Veri Örneklem Boyutu

Sorun: İlk denemede 11 müşteri Gold tablosunda (yetersiz).

Çözüm: Producer 100K mesaj akıttı, ~317 müşteri Gold'a girdi.

9.5 Linear Modellerin Felaket R² Vermesi

Sorun: İlk eğitimde R² = -115000 (outlier'lar nedeniyle).

Çözüm: %99 quantile ile outlier temizliği, R² 0.66'ya çıktı.

10. Sonuç

10.1 Başarılan Hedefler

- Uçtan uca production-grade pipeline kuruldu
- Medallion mimarisi (Bronze/Silver/Gold) uygulandı
- 5 model MLflow ile eğitildi ve karşılaştırıldı
- İnteraktif dashboard ile sonuçlar görselleştirildi
- Pipeline modüler — her bileşen bağımsız çalışabiliyor

11. Teknoloji Stack'i

Katman	Teknoloji	Sürüm
Konteynerizasyon	Docker, Docker Compose	latest
Streaming	Apache Kafka (Confluent)	7.5.0
İşleme	Apache Spark	3.5.1
Depolama	Delta Lake	3.2.0
ML	Spark MLlib	3.5.1
Deney Takibi	MLflow	2.13.0
Dashboard	Streamlit + Plotly	1.35 + 5.22
Dil	Python	3.10
Geliştirme	JupyterLab, VS Code	-

12. Referanslar

- UCI ML Repository — Online Retail II Dataset
- Apache Spark Documentation
- Delta Lake Documentation
- MLflow Documentation
- Streamlit Documentation
- "Medallion Architecture" — Databricks Blog

Proje Repo: <https://github.com/TunaKomur/big-data-online-retail-pipeline>

Commit Sayısı: 13+

Toplam Kod: ~3500+ satır Python + SQL + Markdown